



Figure 3: Precompilation process



Figure 4: Precompiled functions on Firebug

```
((_t=( course ))==null?'':_t)+
' students</h1>';
}
return __p;
```

As I have stated, in template settings, this function uses the *with* statement to control the scope. If we use the *variable* parameter while configuring, this function will be generated without the *with* statement as below:

```
var _t,
    _p='',
    _j=Array.prototype.join,
    print=function(){
        _p+=j.call(arguments,'');
    };

__p+="

# 


```

If you look closely at the above function, you will understand the basic behaviour of the Underscore template. This function creates a logical string concatenation to create the HTML content. It also defines the *print* function.

Modular development

In the above example, the template is kept in the same JavaScript file. To keep the code clean and modular, it can be kept in a separate file. There are many ways to separate the code.

1. Keep the template content as a string and assign it to a global variable. Save this as a .js file and include it by using the `<script>` tag. Use the global variable to access the template content. Make sure that the template file is

included before the JS file where the template is accessed.

2. Keep the template content in a text file. Write a simple server-side script to read the file and echo the content. Write a client-side script to get it by using the AJAX method and keep it in the cache. This will be a very simple and lightweight solution.
3. Use a modular script loader like *RequireJS*. This will help give the whole project more modularity. Here's how you can improve the performance:

- If the number of templates is very few, we can use the first method.
- If many template files exist, we can *minify* the JS files. But, if you want to keep it more optimal and lightweight by loading only the necessary JS files, use the second or third methods.



Note: We can also reduce the initial page loading time by fetching template files on demand. Usually, there is a limitation on the number of files that can load in parallel. If we load all the files first, more time will be taken and will serve a blank page to the user. To avoid this, we can load only the necessary JS files on demand.

Loading templates by using Require.js

Require.js is a module loader. It loads the necessary libraries, HTML or text files, on demand. Explaining the Require.js functionality is not within the scope of this article. But you can see the basic usage of Require.js in the following example.

Get Require.js from <http://requirejs.org/docs/download.html>. It also needs the plugin *text.js* to handle the text or template files. Download it and keep it in the same directory.



Figure 5: Underscore with Require.js



Figure 6: Template on demand